Universidad Externado de Colombia | DEPARTAMENTO DE MATEMÁTICAS

Ciencia de Datos

Estructura de Datos
Parcial II
15 abril 2026

Nombre:	*Documento:*

Ejercicio 1. (1.5 puntos)

✓ Escribir una función *construir(L)* que recibe una lista y retorna el *Linked Binary Tree* que representa.

Ejercicio 2. (1.5 puntos) Con una cola de prioridad se puede completar un queue. Completar la documentación dada a continuación para construir dicha estructura.

```
class FIFOQueue:
    """FIFO queue implemented using heapq and a time counter."""
    def __init__(self):
        self._data = HeapPriorityQueue()
        self._key = 0

    def is_empty(self):
    """Return true if Queue is empty"""



    def enqueue(self, e):
        """Add element e to the back of the queue."""




    def first(self):
        """Return (but do not remove) the element at the front.
        If Queue is empty raise and Exception
        """



    def dequeue(self):
        """Remove and return the element at the front.
        If Queue is empty raise an Exception
        """
```

Ejercicio 3. (1 punto) Tenemos el siguiente código en python:

```
PQ = HeapPriorityQueue()
for i in [1,2,3,4,-5, 6, 0]:
    PQ.add(i,i)

PQ.remove_min(), PQ.remove_min()
```

Dibujar el heap (árbol binario) que queda después de ejecutar el código anterior.

Ejercicio 4. (1 punto) Dibujar la tabla de hash de 11 entradas resultante al usar la función de

$$h(i) = (3i + 5)\, mod\, 11$$

en los keys $12, 44, 13, 88, 23, 94, 11, 39, 20, 16, 5$ asumiendo que las colisiones se manejan usanado chaining (es decir contruyendo edificios). Hacer lo mismo suponiendo que se utiliza linear probing (es decir se ocupa el espacio horizontal)